

5X0

Five in a Row — Prezentare Tehnică Detaliată

Arhitectura aplicației, clasele Python, fluxul logic de funcționare și pipeline-ul CI/CD

1. Introducere

5XO este o aplicație web care implementează jocul „X și 0” (Five in a Row) pe o tablă de 50×50 celule, unde câștigă primul jucător care aliniază 5 simboluri identice (orizontal, vertical sau pe diagonală).

Aplicația este construită folosind o arhitectură client-server clasică:

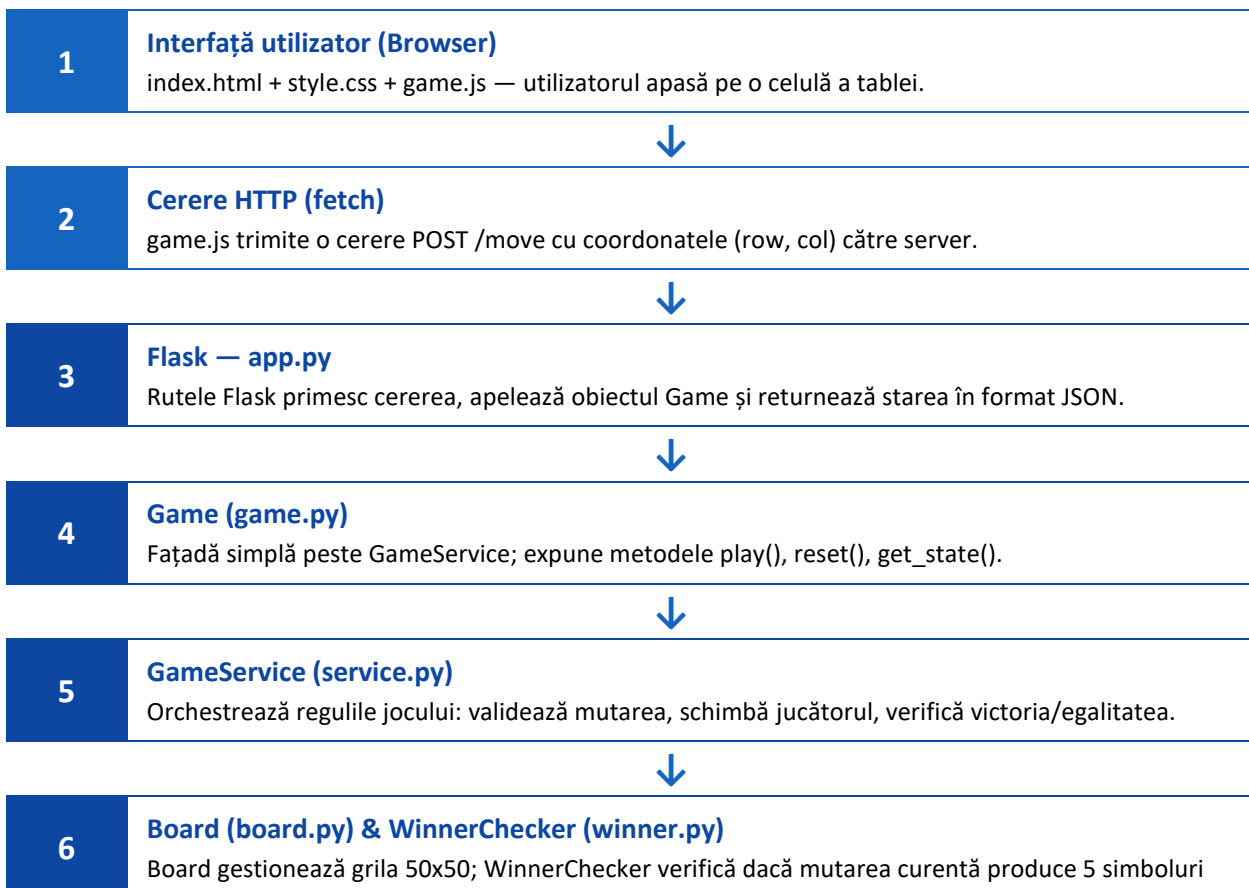
- Backend: Python, framework Flask — gestionează logica jocului și expune un API REST simplu (/state, /move, /reset).
- Frontend: HTML, CSS și JavaScript vanilla — afișează tabla de joc și comunică cu backend-ul prin cereri fetch().
- Containerizare: Docker — aplicația este ambalată într-o imagine Docker pornind de la python:3.13-slim.
- Orchestrare: Kubernetes — aplicația este desfășurată în clusterul Minikube prin resurse Namespace, Deployment și Service.
- Automatizare CI/CD: Jenkins — construiește imaginea, o încarcă în Minikube și redeschășoară aplicația automat.

Documentul de față detaliază rolul fiecărui fișier, funcționarea claselor Python din backend, fluxul logic al unei partide și modul în care funcționează pipeline-ul CI/CD, pas cu pas.

2. Arhitectura generală a aplicației

Aplicația respectă principiul separării responsabilităților (separation of concerns). Logica jocului este complet izolată de stratul web (Flask) și de interfața grafică (HTML/JS), fiind organizată într-un pachet Python dedicat, game/.

2.1 Diagrama arhitecturii



aliniate.

Răspunsul (noua stare a jocului) parcurge fluxul invers, până înapoi la browser, unde game.js actualizează vizual tabla.

3. Structura fișierelor și rolul fiecăruia

3.1 Backend Python (pachetul game/)

Fișier	Rol
__init__.py	Marchează directorul game/ ca pachet Python, permițând importuri de tipul „from game.board import Board”.
game.py	Clasa Game — punct de intrare simplu (fațadă) folosit de app.py; delegă toate operațiile către GameService.
service.py	Clasa GameService — „creierul” aplicației; conține regulile jocului: cine mută, verificarea victoriei, egalitatea, resetarea.
board.py	Clasa Board — reprezintă tabla de joc 50x50 sub formă de matrice; validează poziții și plasează simboluri.
winner.py	Clasa WinnerChecker — verifică, după fiecare mutare, dacă există 5 simboluri identice aliniate pe cele 4 direcții posibile.

3.2 Nivelul web (Flask)

Fișier	Rol
app.py	Aplicația Flask; definește rutele HTTP „/”, „/state”, „/move” și „/reset” și face legătura dintre browser și obiectul Game.
index.html	Template Jinja2 — structura paginii (panou stânga cu informații, tabla de joc, panou dreapta cu butonul de Restart).
style.css	Stilizarea vizuală: grila de 50x50 celule, culorile pentru X/O, evidențierea ultimei mutări și a celulelor câștigătoare.
game.js	Logica din browser: construiește grila de 2500 de celule, trimite mutările către server (fetch) și actualizează interfața pe baza răspunsului JSON.
requirements.txt	Lista dependențelor Python necesare aplicației (Flask ≥ 3.1).

3.3 Containerizare, orchestrare și CI/CD

Fișier	Rol
Dockerfile (aplicație)	Construiește imaginea Docker a aplicației 5XO, pornind de la python:3.13-slim, instalează dependențele și pornește app.py pe portul 5000.
namespace.yaml	Manifest Kubernetes care creează namespace-ul dedicat „fivexo”, izolând resursele aplicației în cluster.

Fișier	Rol
deployment.yaml	Manifest Kubernetes Deployment — descrie cum rulează containerul 5xo:latest (1 replică), sondele readiness/liveness și limitele de resurse (CPU/memorie).
service.yaml	Manifest Kubernetes Service de tip NodePort — expune aplicația în afara clusterului, pe portul 30080, direcționând traficul către podurile cu eticheta app=fivexo.
Jenkinsfile	Definește pipeline-ul CI/CD: build imagine, încărcare în Minikube, aplicare manifeste Kubernetes, redeschimbare și verificare status cluster.
Dockerfile (Jenkins)	Construiește o imagine Jenkins personalizată, cu Docker CLI, kubectl, Minikube și plugin-urile necesare deja instalate.
docker-compose.yml	Rulează containerul Jenkins local, montând socket-ul Docker și configurația kubectl/Minikube de pe mașina gazdă, pentru ca Jenkins să poată construi imagini și comunica cu clusterul.
plugins.txt	Lista plugin-urilor Jenkins instalate automat la build-ul imaginii (Git, GitHub, Pipeline, Docker Workflow, Blue Ocean etc.).

4. Clasele Python — analiză detaliată

Backend-ul aplicației este structurat în 4 clase, fiecare cu o singură responsabilitate clară.

4.1 Clasa Game (game.py)

Rol: Fațadă (facade) — simplifică interacțiunea dintre stratul web (Flask) și logica internă a jocului.

Attribute: service — o instanță de GameService, creată la inițializare.

Metode

Metodă	Descriere
<code>__init__()</code>	Creează o instanță nouă de GameService la pornirea aplicației.
<code>play(row, col)</code>	Deleagă mutarea către <code>service.play(row, col)</code> .
<code>reset()</code>	Deleagă resetarea jocului către <code>service.reset()</code> .
<code>get_state()</code>	Deleagă obținerea stării curente către <code>service.get_state()</code> .

Această clasă nu conține logică proprie — rolul ei este să ofere un API curat pentru `app.py`, ascunzând detaliile de implementare din GameService (design pattern Facade).

4.2 Clasa GameService (service.py)

Rol: Componenta centrală a jocului — coordonează tabla (Board), verificarea câștigătorului (WinnerChecker) și starea partidei.

Attribute: board (instanță Board), current_player („X” sau „O”), game_over (bool), winner (simbolul câștigător sau None).

Metode

Metodă	Descriere
<code>reset()</code>	Repornește tabla, resetează jucătorul curent la „X”, game_over la False și winner la None.
<code>switch_player()</code>	Alternează jucătorul curent între „X” și „O” după fiecare mutare validă.
<code>is_draw()</code>	Parcurge toată grila; returnează True dacă nu mai există nicio celulă liberă (egalitate).
<code>play(row, col)</code>	Metoda principală: validează că jocul nu s-a terminat, plasează simbolul pe tablă, verifică victoria prin WinnerChecker, verifică egalitatea, apoi schimbă jucătorul.
<code>get_state()</code>	Returnează un dicționar cu tabla, jucătorul curent, statusul de final de joc și câștigătorul — folosit direct pentru răspunsul JSON al API-ului.

Logica metodei play() — pas cu pas

1. Dacă `game_over` este True, se ridică o excepție `ValueError("Game finished.")` — nu se mai acceptă mutări.
2. Se apelează `board.place_symbol(row, col, current_player)`, care validează poziția și plasează simbolul.

3. Se apelează `WinnerChecker.check(board, row, col)` pentru a verifica dacă această mutare produce o aliniere de 5.
4. Dacă da → `game_over = True`, `winner = current_player`, metoda se oprește (return).
5. Dacă nu, se verifică `is_draw()`; dacă tabla e plină → `game_over = True` (fără câștigător).
6. Altfel, se apelează `switch_player()` și tura trece celuilalt jucător.

4.3 Clasa Board (board.py)

Rol: Reprezintă structura de date a tablei de joc — o matrice 50x50.

Attribute: `SIZE = 50` (constantă de clasă), `grid` — o listă de liste (matrice) inițializată cu `None` în fiecare celulă.

Metode

Metodă	Descriere
<code>reset()</code>	Recreează grid ca o matrice 50x50 goală (toate celulele <code>None</code>).
<code>is_inside(row, col)</code>	Verifică dacă o poziție se află în limitele tablei ($0 \leq \text{row}, \text{col} < 50$).
<code>is_empty(row, col)</code>	Verifică dacă celula respectivă este liberă (<code>None</code>).
<code>place_symbol(row, col, symbol)</code>	Plasează „X” sau „O” pe celulă, după ce validează că poziția este validă și liberă; altfel ridică <code>ValueError</code> .

Board este o clasă „pură” de date — nu cunoaște nimic despre reguli de câștig sau despre jucători; se ocupă exclusiv de integritatea grilei.

4.4 Clasa WinnerChecker (winner.py)

Rol: Verifică, după fiecare mutare, dacă simbolul plasat formează o linie de minimum 5 simboluri consecutive.

Attribute: `DIRECTIONS` — un tuplu cu 4 perechi (`dr`, `dc`) reprezentând cele 4 direcții posibile de aliniere: orizontală, verticală și cele două diagonale.

Metode

Metodă	Descriere
<code>check(board, row, col)</code>	Metodă de clasă principală: pentru fiecare din cele 4 direcții, numără simbolurile consecutive în ambele sensuri (înainte și înapoi) și verifică dacă totalul ajunge la 5.
<code>count_direction(board, row, col, dr, dc, symbol)</code>	Metodă statică — pornind din celula dată, avansează pas cu pas în direcția (<code>dr</code> , <code>dc</code>) atâta timp cât celulele conțin același simbol și sunt în interiorul tablei; returnează numărul de simboluri identice găsite.

Logica algoritmului de verificare a victoriei

Pentru fiecare mutare nou efectuată, algoritmul verifică 4 axe de aliniere posibile:

- Orizontală: (0, 1) — stânga / dreapta
- Verticală: (1, 0) — sus / jos
- Diagonala „\”: (1, 1) — colț stânga-sus către colț dreapta-jos
- Diagonala „/”: (1, -1) — colț dreapta-sus către colț stânga-jos

Pentru fiecare axă, se pornește de la 1 (simbolul plasat) și se adaugă numărul de simboluri identice găsite mergând într-o direcție (de exemplu la dreapta), plus numărul găsit mergând în direcția opusă (la stânga). Dacă suma ajunge la 5 sau mai mult pe oricare dintre cele 4 axe, jocul se termină cu o victorie.

4.5 Diagrama relațiilor dintre clase

Clasă	Depinde de / Folosește	Este folosită de
Game	GameService	app.py
GameService	Board, WinnerChecker	Game
Board	— (structură de date independentă)	GameService, WinnerChecker
WinnerChecker	Board (doar citire)	GameService

5. Fluxul logic complet al aplicației (rulare end-to-end)

5.1 Pornirea aplicației

7. `app.py` creează instanța globală `game = Game()`.
8. `Game.__init__()` creează un `GameService`, care la rândul lui creează un Board gol (50x50) și setează `current_player = "X"`.
9. Ruta „/” randează `index.html`, care încarcă `game.js` și `style.css`.
10. La încărcarea paginii, `game.js` apelează `createBoard()` (generează cele 2500 celule) și `loadGameState()` (cere GET /state pentru starea inițială).

5.2 O mutare (click pe celulă)

11. Utilizatorul apasă pe o celulă → `game.js` apelează `sendMove(row, col)`.
12. Se trimite POST /move cu {row, col} în corpul cererii JSON.
13. `app.py` extrage row și col și apelează `game.play(row, col)`.
14. `Game` delegă către `GameService.play()`, care validează, plasează simbolul pe Board și verifică victoria/egalitatea prin `WinnerChecker`.
15. Dacă mutarea este invalidă (celulă ocupată, poziție în afara tablei sau joc terminat), se ridică `ValueError`, iar `app.py` răspunde cu status HTTP 400 și un mesaj de eroare, afișat de frontend printr-un `alert()`.
16. Dacă mutarea este validă, `app.py` răspunde cu JSON-ul stării complete a jocului (`get_state()`).
17. `game.js` primește răspunsul, actualizează obiectul local `game` și repictează tabla (`renderBoard()`), colorând simbolurile X/O și actualizând textul de status (jucător curent, câștigător sau egalitate).

5.3 Resetarea jocului

18. Utilizatorul apasă butonul „Restart Game”.
19. `game.js` trimite POST /reset.
20. `app.py` apelează `game.reset()`, care golește tabla și restabilește starea inițială (X începe, fără câștigător).
21. Se returnează noua stare, iar interfața este redesenată complet.

6. Pipeline-ul CI/CD (Jenkins)

Pipeline-ul este definit declarativ în Jenkinsfile și rulează automat la fiecare execuție (ex: la un push în repository sau declanșare manuală din Jenkins). Scopul lui este să construiască imaginea Docker a aplicației, să o încarce în clusterul Minikube și să redesfășoare aplicația în Kubernetes, fără intervenție manuală.

6.1 Infrastructura Jenkins folosită de pipeline

Înainte de a analiza etapele pipeline-ului, este important de menționat cum este pregătit mediul Jenkins:

- **Dockerfile (Jenkins):** construiește o imagine Jenkins personalizată (jenkins-devops:2.0) pornind de la jenkins/jenkins:its, la care se adaugă Docker CLI, kubect!, Minikube și pachetele Python necesare.
- **plugins.txt:** instalează automat plugin-urile Jenkins necesare — integrare Git/GitHub, pipeline-uri declarative, vizualizare Blue Ocean, gestionare credențiale și autorizare pe roluri.
- **docker-compose.yml:** pornește containerul Jenkins, montând socket-ul Docker (/var/run/docker.sock) pentru a putea construi imagini, și configurația kubect!/Minikube a mașinii gazdă, pentru a putea comunica cu clusterul Kubernetes local.

6.2 Schema logică a pipeline-ului



**8**

Cluster Status

Afișează starea finală: podurile, serviciile, deployment-urile și evenimentele recente din namespace-ul „fivexo”, pentru verificare vizuală rapidă.

6.3 Explicație detaliată pentru fiecare stage

Stage 1 — Checkout

Primul pas al oricărui pipeline Jenkins standard: comanda `checkout scm` clonează codul sursă din repository-ul configurat (de exemplu Git), aducând în workspace-ul Jenkins cea mai recentă versiune a codului, inclusiv Jenkinsfile-ul, fișierele sursă Python, manifestele Kubernetes și Dockerfile-ul.

Stage 2 — Environment

Acest stage nu modifică nimic; rolul lui este pur informativ și de depanare (debugging). Se afișează: utilizatorul curent și directorul de lucru, versiunile uneltelor instalate (docker, kubectl, git, python3), contextul Kubernetes activ (kubectl config current-context), informații generale despre cluster (kubectl cluster-info) și lista nodurilor disponibile (kubectl get nodes). Astfel, în cazul unei erori ulterioare, se poate verifica rapid dacă mediul era configurat corect.

Stage 3 — Build Docker Image

Se construiește imaginea Docker a aplicației folosind comanda `docker build -t 5xo:latest .`, pe baza Dockerfile-ului aplicației (python:3.13-slim + Flask). Această imagine este construită în demonul Docker al mașinii gazdă (agentul Jenkins), nu încă în cel al Minikube.

Stage 4 — Load Image into Minikube

Deoarece Minikube rulează propriul demon Docker izolat, imaginea construită la pasul anterior nu este vizibilă automat în interiorul clusterului. Acest stage rezolvă problema astfel:

- `eval $(minikube docker-env)` — comută variabilele de mediu Docker către demonul intern al Minikube.
- `docker build -t 5xo:latest .` — reconstruiește imaginea, de această dată direct în interiorul Minikube.
- `eval $(minikube docker-env -u)` — revine la demonul Docker al mașinii gazdă.
- `docker exec minikube ctr -n=k8s.io images ls | grep 5xo` — verifică, din interiorul containerului Minikube, că imaginea a fost efectiv importată cu succes.

Acest mecanism explică și de ce `deployment.yaml` folosește `imagePullPolicy: Never` — Kubernetes nu trebuie să încerce să descarce imaginea dintr-un registry extern, deoarece ea există deja local, în Minikube.

Stage 5 — Deploy Namespace

Se aplică manifestul `kubectl apply -f k8s/namespace.yaml`, care creează namespace-ul dedicat „fivexo”. Namespace-ul izolează logic toate resursele aplicației (pod, service, deployment) de restul resurselor din cluster.

Stage 6 — Deploy Application

Se aplică, în ordine, manifestele `deployment.yaml` și `service.yaml`:

- Deployment: definește 1 replică a podului 5xo, pe baza imaginii 5xo:latest, portul 5000, sondele `readinessProbe` / `livenessProbe` (verifică dacă aplicația este pregătită să primească trafic, respectiv dacă rulează în continuare corect) și limitele de resurse (CPU/memorie).

- Service: de tip NodePort, expune aplicația din interiorul clusterului către exterior, mapând portul intern 5000 la portul extern 30080, direcționând traficul către podurile cu eticheta app=fivexo.

Aplicarea acestor manifeste este idempotentă — dacă resursele există deja, Kubernetes le actualizează; dacă nu există, le creează.

Stage 7 — Restart Deployment

Chiar dacă manifestele nu s-au schimbat structural, imaginea 5xo:latest s-ar putea să fi fost reconstruită cu cod nou (același tag, conținut diferit). Din acest motiv, pipeline-ul forțează o repornire explicită a podurilor cu `kubectrl rollout restart deployment/fivexo -n fivexo`, garantând că podurile noi vor folosi cea mai recentă imagine încărcată în Minikube. Comanda `kubectrl rollout status` așteaptă apoi, cu un timeout de 180 de secunde, ca noile poduri să devină „Ready”, oprind pipeline-ul cu eroare dacă rollout-ul eșuează sau durează prea mult.

Stage 8 — Cluster Status

Ultimul stage afișează, în consola Jenkins, o imagine de ansamblu a stării finale a namespace-ului „fivexo”: lista podurilor (cu detalii extinse -o wide), serviciile, deployment-urile și evenimentele recente sortate cronologic. Acest raport ajută la verificarea rapidă, fără a fi nevoie de acces direct la cluster, că desfășurarea a reușit.

6.4 Gestionarea erorilor (post actions)

Blocul post din Jenkinsfile definește acțiuni care rulează întotdeauna la finalul pipeline-ului, în funcție de rezultat:

- success: afișează un mesaj simplu de confirmare că desfășurarea s-a încheiat cu succes.
- failure: colectează automat informații extinse de depanare — starea tuturor resurselor din namespace (`kubectrl get all`), detalii despre Deployment și despre poduri (`kubectrl describe`), log-urile complete ale containerelor aplicației (`kubectrl logs`) și evenimentele recente din cluster — toate afișate direct în consola Jenkins, pentru diagnosticare rapidă a cauzei eșecului.
- always: rulează `cleanWs()`, care curăță complet workspace-ul Jenkins, indiferent de rezultatul pipeline-ului, pentru a evita acumularea de fișiere temporare între execuții succesive.

6.5 Configurări suplimentare ale pipeline-ului

Opțiune	Efect
<code>timestamps()</code>	Adaugă marcaje de timp fiecărei linii din log-ul consolei Jenkins.
<code>disableConcurrentBuilds()</code>	Împiedică rularea simultană a mai multor build-uri pentru același job, evitând conflicte pe cluster/imagine.
<code>buildDiscarder(logRotator(numToKeepStr: '10'))</code>	Păstrează doar ultimele 10 execuții ale pipeline-ului, ștergând automat istoricul mai vechi.

7. Concluzii

Aplicația 5XO este structurată pe straturi clar delimitate: o logică de joc pură și testabilă (pachetul game/), un strat web subțire (Flask) care expune această logică prin HTTP, o interfață simplă (HTML/CSS/JS) și o infrastructură completă de containerizare, orchestrare și livrare continuă.

Separarea responsabilităților în cele 4 clase Python (Game, GameService, Board, WinnerChecker) face codul ușor de întreținut și de extins — de exemplu, regulile de câștig din WinnerChecker pot fi modificate fără a afecta reprezentarea tablei din Board sau fluxul general din GameService.

Pipeline-ul CI/CD automatizează complet ciclul „cod nou → imagine Docker → redesfășurare în Kubernetes”, oferind în același timp rapoarte detaliate de stare și de eroare, ceea ce reduce semnificativ timpul necesar identificării problemelor în producție.